JC4004 – Computational Intelligence

Sessions 46-48: Wrap-up

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

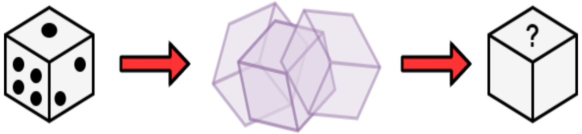
Oct-Dec 2025

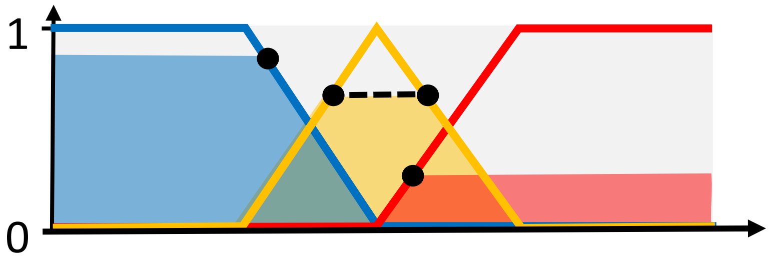
Wrap-up: an overview

- This course covers the essential topics in computational intelligence
  - Computational intelligence is an area of artificial intelligence (AI), with focus on biologically and linguistically inspired learning-based methods
  - Conventionally, the three main areas of computational intelligence are fuzzy logic, genetic algorithms, and artificial neural networks
  - Many topics in computational intelligence have probabilistic elements
- Assessment based on assignment (30%) and exam (70%)
  - Coursework tests the ability to work as a group and use knowledge of computational intelligence to solve a practical problem (game bot)
  - Exam tests the general understanding of the course material

1-14: classic computational intelligence

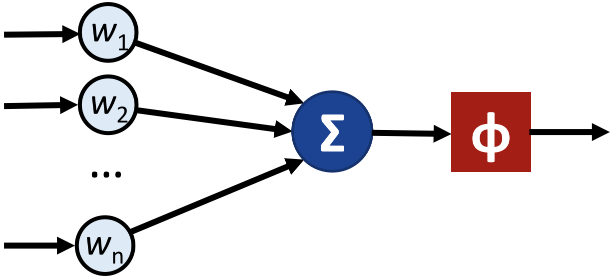
- The main theme of Sessions 1-14 is to cover the traditional topics within computational intelligence
  - Basic probability theory as it is required for understanding many other topics, such as Gaussian processes and hidden Markov models
  - Along with artificial neural networks, fuzzy logic and genetic algorithms form the basis for modern computational intelligence

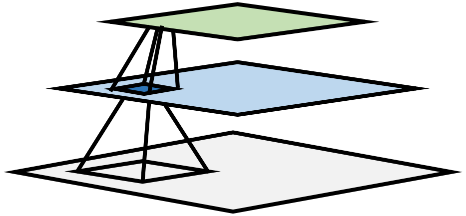




15-28: artificial neural networks

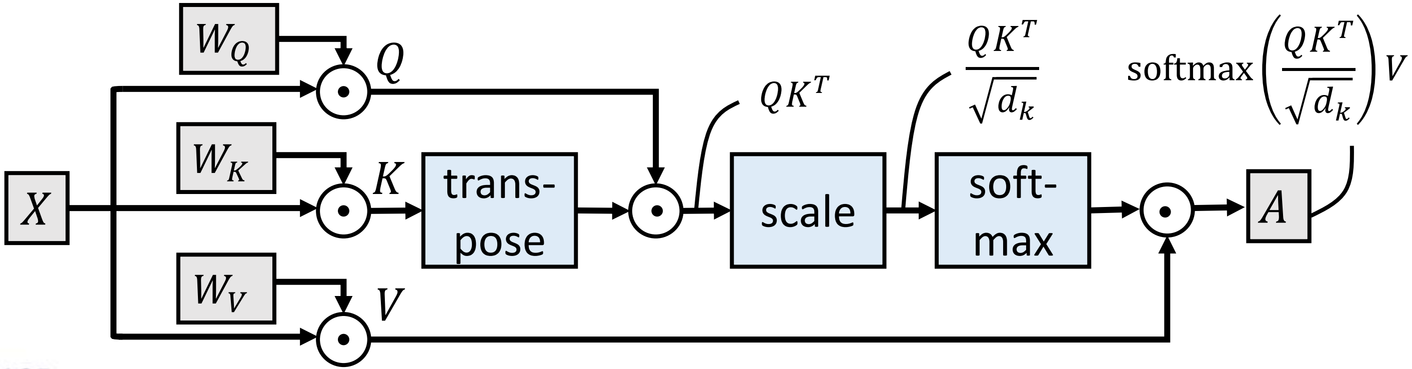
- Sessions 15-28 cover the basics of artificial neural networks, including the fundamentals of deep learning
    - The basic architectures for modern neural networks, including convolutional neural networks and ResNets, introduced
    - Initialisation and training of neural networks, as well as regularization techniques, also discussed





29-38: modern deep learning

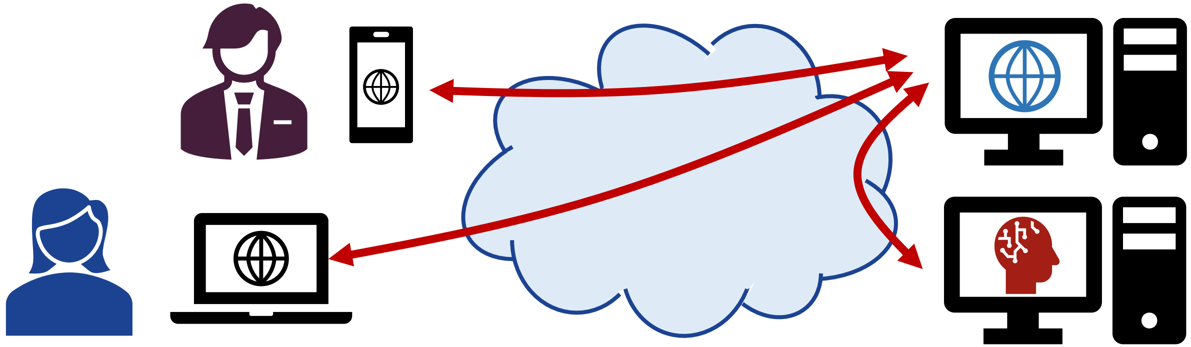
- Sessions 29-38 the advanced deep learning architectures forming the basis for the modern artificial intelligence
  - Attention and transformers are the major advances in neural networks since deep convolutional neural networks, including residual networks
  - Structured state-space models for sequences for even one step further



UNIVERSITY OF ABERDEEN

39-45 generative and multimodal AI

- In sessions 39-45, trending models for generative and multimodal AI have been introduced
  - Basis for the ongoing AI hype, including large language models and generative models for realistic image synthesis
  - Some ongoing trends also covered briefly (green AI, ethical AI)



UNIVERSITY OF ABERDEEN

The exam

- The “small” questions (50%-60% of the exam):
  - The “small” questions can be e.g., multiple choice questions, true/false statements, fill in terms, combining terms to explanations
  - The “small” questions cover all the lectures
  - Some questions are easy, some of them more difficult
- The “big” questions (40%-50% of the exam):
  - The “big” questions require broader knowledge of a certain topic and can include more complex problem-solving elements
  - The “big” questions will focus on specific topics
  - Can comprise smaller questions concerning the same topic/problem

Example questions: MSC 1

- Which of the following suits the best for predictions from sequential data?
  A) Ant colony optimisation (ACO)
  B) Recurrent neural network (RNN)
  C) Convolutional neural network (CNN)
  D) Feed-forward neural network (FNN)

Example questions: MSC 2

- Given ground truth vector [0,0,1,0] and prediction [0.1,0.2,0.3,0.4], what is the cross-entropy loss?
  A) $-0.3 \cdot \log(0.7)$
  B) $\log(0.7)-\log(0.3)$
  C) $-\log(0.3)$
  D) $\log(0.3)$

UNIVERSITY OF ABERDEEN

Example questions: MSC 3

- Which of the following is true about self-attention?
  A) Q, K, and V matrices always have the same dimensions.
  B) Q, K, and V matrices can have the same or different dimensions.
  C) Q and K matrices always have the same dimensions, V matrix can have the same or different dimensions.
  D) Q, K, and V matrices always have different dimensions.

Example questions: True/False

- Backpropagation is a weight optimisation algorithm.
- When values 0 and 1 are used, fuzzy operators are consistent with Boolean operators.
- Multilayer perceptron was invented in 1970s.
- In GANs, generator aims to minimise loss and discriminator aims to maximise loss.

Example questions: Combinations

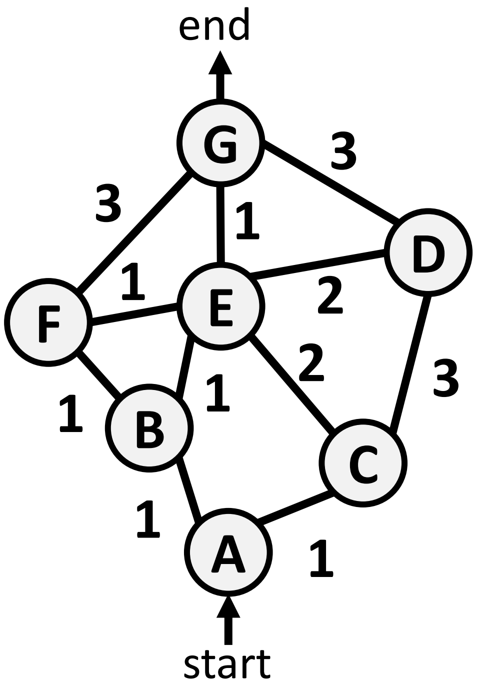
- Connect terms A-C to correct locations 1-3: A) swarm; B) cosine; C) dilated
  - In convolutional neural networks, perceptive field can be expanded by using ____1______ convolutions.
  - Collaboration of simple agents to complete a complex task is called _____2______ intelligence.
  - Similarity of two distributions can be measured by using e.g., _____3______ similarity.

The "big" question topic areas

- The "big" questions could concern any of the following topics:
  - Game theory (sessions 5&6)
  - Hidden Markov models (sessions 7&8)
  - Fuzzy logic (sessions 9&10)
  - Genetic algorithms (sessions 11&12)
  - Swarm intelligence (sessions 13&14)
  - Artificial neural networks (sessions 15&16, 21&22)
  - Re-inforecement learning (sessions 17&18)
- Knowledge of probability theory (sessions 3&4) can also be required

1495 UNIVERSITY OF ABERDEEN

Example "big" question: swarm intelligence



- Given the graph to the left (the numbers denote distances), answer the questions about Ant Colony Optimisation (ACO):
  - Given that three first ants follow routes A-B-F-E-G, A-C-D-G, and A-C-E-F-G, and global pheromone update is used, which of the ants leaves the strongest pheromone trail?
  - Another ant enters the trail after the three. Which route it is most likely to follow? We assume that in the transition rule, $\alpha, \beta > 0$.

Example “big” question: HMM

- Given the Python code for hidden Markov model, answer the questions:
  - Write transition matrix A and observation matrix B
  - If starting from hidden state 1, what is the probability of observed state sequence {X,Y,X}?

```
import random as r

def hmm(hidden_state, steps):
  if steps == 0:
    return
  if hidden_state==0:
    if r.random() < 0.5:
      print('X') # Observed X
    else:
      print('Y') # Observed Y
    if r.random() < 0.3:
      hmm(0, steps-1)
    else:
      hmm(1, steps-1)
  else:
    print('X')  # Observed X
    if r.random() < 0.4:
      hmm(0, steps-1)
    else:
      hmm(1, steps-1)
```

That’s all, folks. Questions?

UNIVERSITY OF ABERDEEN